The Junction Linked List (JLI): An Externally Indexed Sorted Linked List and an Empirical Comparison with the Skip List

Author : Mehal Chaudhary **Date** : Nov 2, 2026

Email : mehal17chaudhary@gmail.com

Github : https://github.com/mehal17chaudhary-sudo/

Abstract

Sorted in-memory dictionaries usually pay for fast search by storing index pointers inside every node. The Junction Linked List (JLI, also called the AMJ List) removes them: keys live in a bare sorted singly linked list, and search structure is held in separate external layers — junctions over fixed-size segments, blocks of junctions, and a block-level skip list. Because the nodes know nothing about the index, a mutation can leave it slightly stale, and deferred three-tier maintenance (local repair, region rebuild, global rebuild) bounds how stale it can become. We compare JLI with a textbook skip list (p = 0.5) implemented in the same C translation unit and timed by the same harness, over 87 configurations (bulk build, static search, insertion and deletion; up to six access patterns; n from 10K to 1M 32-bit keys) on one machine, with a search-only replication on a second machine and a maintenance on/off comparison for insertion. JLI's analytical, payload-free index memory was 75–80 % of the skip list's in every configuration, and bulk construction was 2.1–2.2× faster than building the skip list by repeated insertion. Static search was faster in 25 of 30 configurations on the main machine, by small margins (ratios 0.81–1.23), although its dependence on n was not reproduced on a second machine. Mutation was usually slower (17 of 25 insertion and 19 of 25 deletion configurations, by up to 2.05×), but within 11 % of the skip list in 21 of 25 insertion and 15 of 25 deletion configurations. The maintenance comparison shows that deferral has a break-even horizon: maintenance increases JLI's insertion latency by up to 38 % on short runs, but on long runs it lowers it by up to 23 % for random, zipfian, hotspot and (at the longest run length) sequential insertion, whereas without it JLI's insertion latency drifts to about 1.5× the skip list's. Tail-end workloads are the exception: a tail-sealing rule makes appends cheap independently of maintenance, so maintenance did not help there, and tail deletion, which has no such rule, is JLI's weakest result. Parameters were tuned per workload against the baseline, so the results are best-case for JLI; the study is single-threaded, uses one baseline, and does not support claims of general superiority.

Keywords: sorted linked list; skip list; external index; deferred maintenance; amortised rebuilding; memory efficiency; benchmark methodology.

1 Introduction

Ordered in-memory dictionaries are a basic building block of databases, caches and runtime systems. The skip list [1] is a common choice because it is simple to implement, but it stores its index inside every node: each node carries one forward pointer per level, which averages two pointers at level probability p = 0.5. This index cost is paid per key.

The Junction Linked List (JLI) takes the opposite approach. Keys live in a plain singly linked list whose nodes hold only a key, a payload pointer and a next pointer. Everything needed for fast search is stored *outside* the nodes: a *junction* per fixed-size segment with a small array of shortcut pointers, *blocks* that group consecutive junctions, and a skip list over the blocks (Section 3, Figure 1). The design question we study is therefore: *can an externally indexed sorted list keep search latency competitive with a skip list while using less memory, and what does it cost on mutation?*

Key idea. A skip list keeps its index correct at all times and pays for that on every insertion, in every node. JLI does the opposite: a mutation changes the list and the one junction that owns the touched segment, and the index is allowed to be slightly wrong. This is safe because search never trusts a shortcut blindly: it binary-searches the shortcuts and then walks the few remaining nodes, so a stale shortcut costs a slightly longer walk, not a wrong answer. A tiered maintenance mechanism bounds the staleness, repairing one junction at a time when drift is local, rebuilding a region when it is not, and rebuilding everything only as a last resort. The empirical questions are how much memory this saves, what it does to search, and when the deferred repair pays for itself.

Naming. The structure is also called the *AMJ List* (Anna Mani Junction List), as a tribute to the Indian physicist Anna Mani. This is an alternative name for the same structure, not a separate data structure; we use "JLI" throughout.

Contributions. (i) An externally indexed sorted list in which nodes carry no index data, search tolerates a stale index, and three escalating maintenance tiers bound the staleness (Section 3). (ii) A measured break-even for deferred maintenance: its cost is front-loaded and its benefit appears at longer run lengths, except for tail workloads (Sections 5.4–5.5 and 6.1). (iii) A tail-sealing rule that makes monotonic appends O(1) amortised independently of maintenance, and an analysis showing that its mirror, deletion from the tail, is not covered by it, with measurements on both (Sections 3.2, 3.4 and 5.3). (iv) A benchmark methodology that times JLI and a textbook skip list under the same harness, with raw per-process data retained and every statistic recomputable from it, applied to 87 configurations, a second-machine replication of the search suite and a maintenance on/off comparison, reported with confidence intervals and without per-cell significance claims (Sections 4 and 5). (v) The scripts that regenerate every number and figure from the raw data (Appendix A).

Summary of findings. JLI used 75–80 % of the analytical memory of the skip list in all configurations and built 2.1–2.2× faster than incremental insertion. Its search advantage on the main machine was small and workload-dependent, and its size dependence was not reproduced on a second machine. Insertion and deletion were mostly slower than the baseline, usually modestly, with deletion at the tail the worst case. Maintenance is what keeps the deferred design from degrading: with it off, JLI's insertion latency drifts to about 1.5× the skip list's on long runs; with it on, the same workloads settle lower, at the price of a front-loaded cost. We do not claim that JLI is novel in all its aspects or superior to other ordered indexes; Section 2 places it among related work, and Section 7 lists the limits of the evidence.

2 Background and Related Work

Skip lists and variants. Pugh introduced the probabilistic skip list [1]; Munro, Papadakis and Sedgewick gave deterministic versions [2]. Later work proposes biased skip lists [3], a B-tree-like skip list [4], modified skip lists studied experimentally [5], write-optimised skip lists [6], and designs addressing

concurrency and contention [7], [8], [9]. Our baseline is the unmodified textbook skip list; none of these variants is evaluated here.

Search in sorted lists with auxiliary structure. A long line of work represents sorted or linear lists together with auxiliary structures, including finger-based representations [10], [11], other sorted-list structures [12], searches that exploit locality [13], and order maintenance in linked lists [14]. JLI's junctions play a role similar to such auxiliary structures, but we do not evaluate against them and do not claim a relationship beyond this general family.

Blocked and cache-conscious indexes. B-trees [15] group keys into blocks; cache-conscious main-memory variants [16], [17] and cache-oblivious trees [18] adapt this to the memory hierarchy. Unrolled linked lists store several elements per node [19]. The adaptive radix tree [20] is a main-memory ordered index with a different design point. JLI keeps single-key nodes and places grouping in a separate layer.

Rebuilding and adaptive structures. Scapegoat trees rebuild subtrees when imbalance is detected [21]; splay trees adapt to access patterns [22]; gapped arrays make sorted insertion cheaper [23]. JLI's maintenance tiers (local repair, region rebuild, global rebuild) are in the same spirit of deferring structural repair.

Learned indexes. Learned indexes [24], [25] replace parts of the index with models, and benchmark studies [26], [27] compare them with traditional structures. We do not compare with learned indexes.

Benchmarking methodology. Statistically careful performance evaluation [28], [29], measurement bias [30] and reporting practice [31] motivate our use of repeated independent process runs, interval estimates and explicit threats to validity.

Positioning. We searched the literature with the bounded procedure described in the supplementary search log (OpenAlex plus targeted web checks, top of each ranked list screened). We did not find a work that combines an external segment-level index with shortcut arrays, a block-level skip list and the three-tier deferred maintenance described below. This is the result of a limited search and does not establish novelty; we therefore present JLI as a specific design evaluated empirically against one baseline.

3 The JLI Design

3.1 Structure

A JLI stores n strictly ascending 32-bit integer keys in a singly linked list. A node holds a key, a payload pointer and a `next` pointer (24 bytes with alignment padding). Three layers sit above this base list (Figure 1).

Junctions. The list is partitioned into contiguous segments of about S nodes. Each segment is controlled by an external junction holding the segment's first and last node, its length, an *anchor* node near the middle used as an extra probe point, and a non-decreasing array of K shortcut pointers whose first and last entries are the segment's ends. Shortcuts are placed by repeatedly halving the longest remaining gap.

Blocks. A block is an array of at most M + 2 consecutive junctions (M is the soft capacity). A block does not cache its key range; the range is read from its first and last junction, so in-place edits cannot leave it stale.

Block skip list. A probabilistic skip list (p = 0.5) whose elements are blocks, with geometric levels capped at a maximum derived from n, S and M. It is never patched piecemeal: any change to blocks triggers a single-pass rebuild of its pointers.

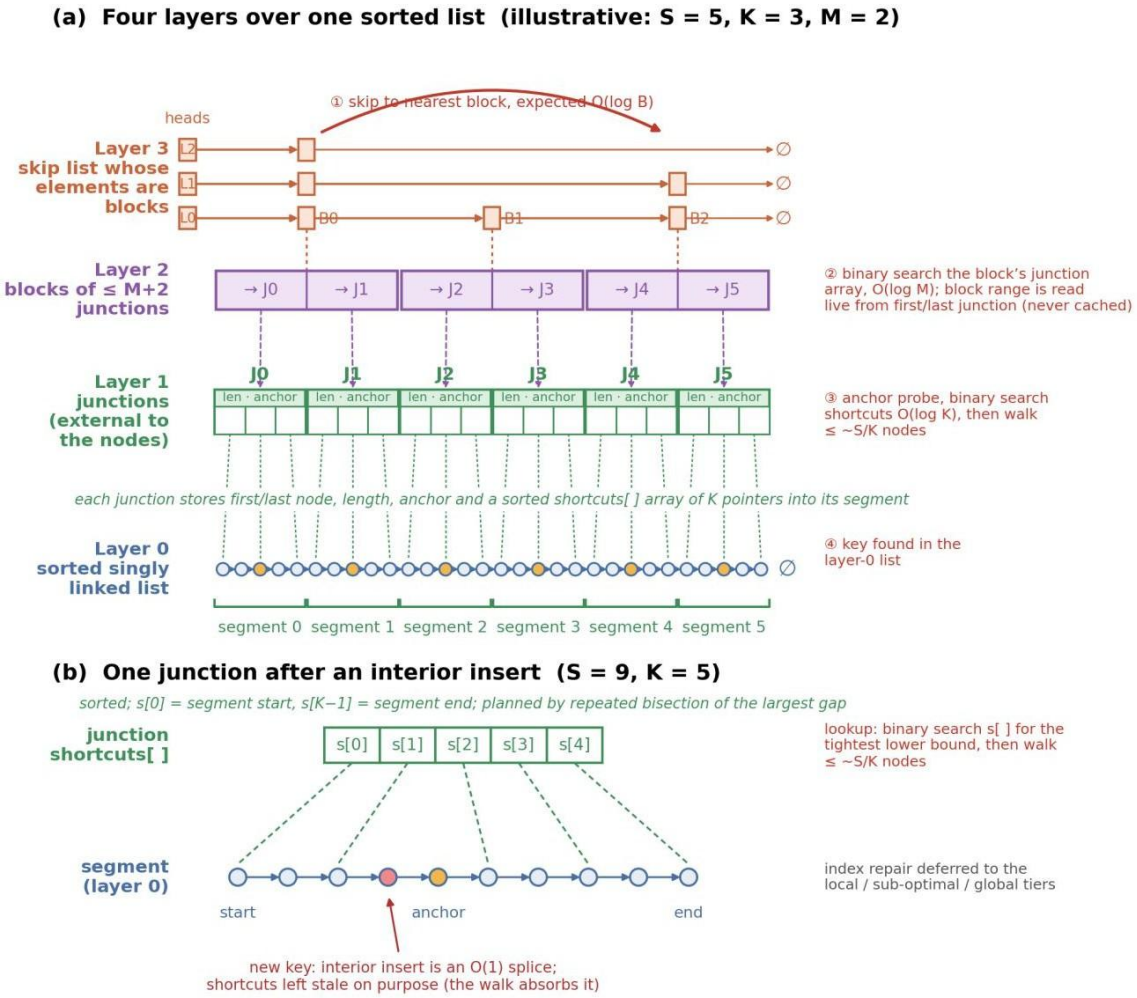


Figure 1. Structure of JLI. (a) Four layers over one sorted list (illustrative S = 5, K = 3, M = 2). Layer 0 is the bare linked list; each junction (layer 1) owns one segment and stores its first and last node, length, an anchor node near the middle, and K shortcut pointers into the segment; a block (layer 2) holds consecutive junctions and reads its key range live from its first and last junction; layer 3 is a skip list whose elements are blocks. Numbers ①–④ trace a lookup. (b) One junction after an interior insertion: the new node is spliced in in O(1) and the shortcuts are deliberately left stale; the final walk absorbs it, and repair is left to the maintenance tiers.

Table 1 lists the symbols. S, K, M and the maintenance parameters are fixed at creation and do not scale with n.

Symbol	Meaning
n	number of keys
S	target segment size
K	shortcut pointers per junction
M	soft block capacity (junctions per block)
$J \approx n/S$	number of junctions
$B \approx J/M$	number of blocks
τ	tolerance of the anchor's position around the segment midpoint
`I_loc` `I_sub`	local and sub-optimal scan intervals (in mutations)
`δ_soft` `δ_hard`	soft and hard segment-size drift thresholds
`ρ_fl` `ρ_em`	flagged-ratio and hard-ratio thresholds for rebuilds
`E_min` `ρ_ss`	region rebuilds and flagged ratio that force a global rebuild

Table 1. Symbols.

3.2 Operations

Search. Search descends the block skip list to find a block whose range contains the key, binary-searches the block's junction array, checks the anchor, binary-searches the junction's shortcut array, and finishes with a linear walk of at most about S/K nodes. A key outside every block range is rejected immediately.

Insert. The splice point is located by the same descent and the new node is linked in O(1). The owning junction's length (and possibly anchor offset) is patched, or the segment boundary updated if the key falls at an end. Interior inserts deliberately leave the shortcut array stale rather than paying $\Theta(S)$ to rebuild it.

Tail sealing. If the owning junction is the last one and its segment reaches S + 3 nodes, the insert splits it: the first S nodes stay as a sealed junction (its shortcuts are re-planned once) and the remaining nodes, at least three, become a new last junction, which is appended to the last block, or to a new last block linked into the block skip list when the last block is full. The slack of 3 ensures that the new tail never starts with two nodes or fewer, which the delete path would dissolve immediately. A seal costs $\Theta(S)$ and occurs about once per S appends, so monotonic appends cost O(1) amortised beyond location, with no scan, region rebuild or wholesale rebuild of the block skip list. Sealing is part of the insert path, not of the maintenance tiers: it is active whether or not maintenance is enabled, including in both arms of the maintenance comparison (Section 4.6). It applies only to the last junction, and deletion from the tail has no counterpart.

Delete. After locating the key, the node is unlinked. If the segment keeps more than two nodes the junction is repaired locally (replacing a deleted anchor or shortcut by a nearby node, or rebuilding the shortcut array in $\Theta(S)$ if that fails). If the segment shrinks to two nodes or fewer, it is merged into a

neighbour or removed, the junction links are rewired and the block skip list is rebuilt before the call returns (always on, independent of maintenance).

3.3 Deferred maintenance

Because nodes carry no index data, the index drifts from its ideal shape under mutation. *Positional drift* moves an anchor away from the segment midpoint, outside a tolerance τ. *Size drift* makes a segment deviate from S by at least `δ_soft` (flagged) or `δ_hard` (hard). After each mutation a hook triggers up to three escalating tiers (Table 2).

Tier	Scan	Repair	Cost
Local	every `I_loc` mutations; offset ratio against τ	re-plan shortcuts of the flagged segment	scan O(J); repair Θ(S)
Sub-optimal	every `I_sub` mutations; size drift per junction	re-segment flagged regions, repack blocks, rebuild block skip list	scan O(J); O(region) + O(B)
Global	checked at each sub-optimal scan	discard index and rebuild over the live list	O(n)

Table 2. Maintenance tiers.

A sub-optimal scan rebuilds all flagged regions if their fraction reaches `ρ_fl`, otherwise only hard junctions. A global rebuild fires if the hard fraction reaches `ρ_em` (emergency) or if at least `E_min` region rebuilds have run since the last global rebuild and the flagged fraction is still at least `ρ_ss` (sustained drift). Configurations with `I_loc` ≥ `I_sub` or `δ_soft` ≥ `δ_hard` are rejected. Tail sealing (Section 3.2) is not one of these tiers and is not controlled by the maintenance switch.

3.4 Complexity

We assume the word-RAM model.

Search. The phases cost O(log B) + O(log M) + O(log K) + O(S/K). With B = n/(SM) and M, K constant, this is O(log n + S/K); it is O(log n) only if K = Θ(S). In all our experiments K is fixed, so the S/K term remains.

Mutation. Given the location, insertion is O(1) and an ordinary deletion is O(1) except when a shortcut is removed (Θ(S)). Segment dissolution costs O(S + J + B) in the worst case.

Amortised cost (informal). The amortised mutation cost is the location cost plus scan costs O(J)/I_loc and O(J)/I_sub plus tier firing rates times their repair costs. Whether this is O(log n) depends on the workload through the firing rates. Under uniformly spread insertions we expect rebuilds to be rare. Without tail sealing, repeated appends at the tail would all land in the rightmost junction, drift would reach the hard threshold every Θ(S) operations, and a region rebuild with its O(B) floor would fire at rate Θ(1/S), adding a cost of order n/(S²M) per operation that grows with n. Tail sealing removes this case for appends: a seal costs Θ(S) once per about S appends, so appends are O(1) amortised beyond location. Its mirror, deletion from the tail, is not covered: each deletion shrinks the last segment from its far end until it is dissolved, at a cost that grows with J + B, once per Θ(S) deletions, so the per-operation cost of tail deletion plausibly grows with n; the measured ratio, 1.31 at 50K to 2.05 at 1M, is in that direction.

This is an informal argument, not a proof, and the benchmark sizes cannot verify an asymptotic rate (Section 6).

Space. A JLI node is 24 bytes. A skip-list node in our baseline has a 16-byte header and one 8-byte pointer per level, 32 bytes on average at p = 0.5. Junction, block and block-skip-list overhead is Θ(n/S) and Θ(n/(SM)). Total index overhead is Θ(n) for both structures; JLI's constant is smaller because the per-node index is moved into shared structures.

4 Experimental Methodology

4.1 Baseline and implementation

The baseline is a textbook probabilistic skip list with p = 0.5, implemented in the same C translation unit as JLI and timed by the same harness, which shares the seeded key pools, the query and operation generators, and the timing and memory-accounting code. Keys are 32-bit integers and no payload is stored (payload pointers are null). The maximum skip level is derived from n (⌈ $\log_2 n$ ⌉ for the skip list, ⌈ $\log_2(n/(SM))$ ⌉ for the block skip list, both clamped to [1, 64]); we checked this derivation against the recorded level for every configuration. The code is compiled with GCC (MSYS2) at `-O2` and contains no profiling instrumentation in the timed binary.

4.2 Hardware

The main suite ran on one machine (machine A: AMD Ryzen 7 7735HS, 8 cores, 16 GB RAM, Windows 11). The search suite (STATIC) was repeated on machine A and run on a second machine (machine B: Intel Core i5-1155G7, 4 cores/8 threads, 8 GB RAM, Windows 11), using the parameters selected in the main suite without further tuning . All runs are single-threaded.

4.3 Benchmark classes, sizes and patterns

Table 3 lists the classes. BUILD is bulk construction of JLI from a sorted array, compared with a skip list built by inserting the keys one by one. STATIC is search only. INSERT and DELETE are mutation workloads with maintenance enabled. For STATIC, INSERT and DELETE the number of operations is ⌊ 2n/7 ⌋ (14 285 at n = 50K to 285 714 at 1M); in the maintenance comparison it is a multiple of n (Section 4.6).

Class	Patterns	Sizes n	Configs
BUILD	sorted	10K, 50K, 100K, 250K, 500K, 750K, 1M	7
STATIC	random, sequential, hotspot, zipfian, miss, adversarial	50K, 100K, 250K, 500K, 1M	30
INSERT	random, sequential, hotspot, zipfian, adversarial	same	25
DELETE	random, sequential, hotspot, zipfian, adversarial	same	25

Table 3. Benchmark classes (7 + 30 + 25 + 25 = 87 configurations).

All patterns draw from one ascending key pool of size n. *random* is uniform over the pool (for INSERT, a new key is synthesised in the gap next to the drawn position, or appended past the maximum). *sequential* visits pool positions in ascending order. *hotspot* draws from the top 10 % of the pool with probability 0.8, otherwise uniformly. *zipfian* weights pool ranks by 1/rank, favouring small keys. *miss* (STATIC only) queries keys guaranteed absent. *adversarial* is defined per class and is not comparable across classes: for STATIC, for each junction the query that costs the most steps inside that junction (shortcut search plus longest linear walk) is chosen and junctions are visited in shuffled order, so all queries are hits present in both structures; for INSERT, successful appends at increasing keys past the maximum, growing the rightmost junction; for DELETE, deletion of pool indices n − 1, n − 2, …, shrinking the tail from its far end. The harness also contains a degenerate "tail miss" pattern (every query above the maximum, answered at the tail range check); it is excluded from all results.

4.4 Parameter selection

BUILD uses one fixed configuration at every size (S = 128, K = 4, M = 4). STATIC, INSERT and DELETE were tuned per (class, pattern) by a hierarchical random search at a 500K-key reference size and then transferred and locally refined at the other sizes. The search covered the core parameters ($S \in [256, 16384]$, $K \in [2, 32]$, block size $\in [2, 24]$) and the maintenance parameters. The objective was the latency ratio against the skip list ; memory was never an objective. Tuning and evaluation used the same benchmark workloads on the same machine, so the reported ratios are *in-sample* best-case estimates for JLI; the skip list was not tuned. The selected parameters are in the supplementary data.

4.5 Measurement protocol

Timing. Wall-clock time (monotonic clock) is measured around a batch of operations. One *internal run* builds fresh, re-seeded structures and times the batch for each structure; the reported per-run value is the mean time per operation, so all latency figures are batch means and not per-operation tail latencies. Each configuration uses 30 internal runs per process. The harness is invoked as 11 independent processes with the first discarded as a cold start, leaving 10 retained processes per configuration. Within an internal run, the timing order of JLI and the skip list alternates with run parity. JLI's step counters are compiled out of the timed binary and the search result is consumed through a volatile sink after the timer stops.

Statistics. All statistics are computed from the raw per-process rows. The per-process ratio is JLI mean latency divided by skip-list mean latency; the reported ratio is its mean over the 10 retained processes with a 95 % t-interval. A configuration is classed "JLI faster" if the interval lies below 1, "JLI slower" if above 1, and "CI spans 1" otherwise. We also computed an exact two-sided Wilcoxon signed-rank test on the 10 paired means; with 10 pairs its smallest attainable p is $2/2^{10} = 0.00195$, so after Holm correction across the 87 main-suite tests no cell can reach 0.05 (smallest adjusted p = 0.17). We therefore do not make per-cell significance claims and report intervals descriptively; raw and Holm-adjusted p-values are listed in Table 9 for transparency.

Memory. Memory is charged analytically by an accounting function that sums allocated bytes (headers, per-level pointer arrays and, for JLI, unfilled growth capacity), using the same rule for both structures. The ratio is therefore payload-free structural overhead; resident-set size was not measured.

4.6 Maintenance comparison

INSERT was run with maintenance on and off at n ∈ {50, 100}K and operations/n ∈ {1, 5, 20, 40, 50} for five patterns, using the parameters tuned for INSERT with maintenance on for both arms. The structure grows with the operations, so the final size is up to 51n. The comparison uses the 10 (n, operations) cells completed for all five patterns in both arms: n = 50,000 with 50,000, 250,000, 1,000,000, 2,000,000 and 2,500,000 operations, and n = 100,000 with 100,000, 500,000, 2,000,000, 4,000,000 and 5,000,000 operations. All completed cells are listed in Appendix C. Tail sealing (Section 3.2) is part of the insert path and was active in both arms.

5 Experimental Results

Ratios are JLI divided by skip list, so values below 1 favour JLI. Table 4 summarises the 87 main-suite configurations.

Class	Cells	JLI faster	CI spans 1	JLI slower	Ratio range
BUILD	7	7	0	0	0.45–0.48
STATIC	30	25	3	2	0.81–1.23
INSERT	25	8	0	17	0.65–1.39
DELETE	25	4	2	19	0.80–2.05

Table 4. Main-suite summary (machine A). Classification uses the 95 % interval of the mean ratio.

5.1 Build time and memory

Bulk construction took 2.1–2.2× less time than building the skip list by repeated insertion in all seven sizes (Table 5, Figure 2); this compares bulk construction with incremental insertion and is not a comparison of equal operations. The analytical memory ratio was 0.75–0.80 in every one of the 87 configurations and 0.78 for BUILD with its fixed parameters (24-byte JLI nodes against a 32-byte average skip-list node, plus shared index structures). Because memory was not a tuning objective, the near-constant ratio across tuned configurations is an observation, not a guarantee.

n	BUILD time ratio [95% CI]	Analytical memory ratio
10000	0.45 [0.44, 0.46]	0.78
50000	0.48 [0.47, 0.48]	0.78
100000	0.47 [0.46, 0.47]	0.78
250000	0.46 [0.45, 0.47]	0.78

500000	0.45 [0.45, 0.46]	0.78
n	BUILD time ratio [95% CI]	Analytical memory ratio
750000	0.46 [0.45, 0.47]	0.78
1000000	0.45 [0.43, 0.46]	0.78

Table 5. BUILD time and analytical memory ratios.

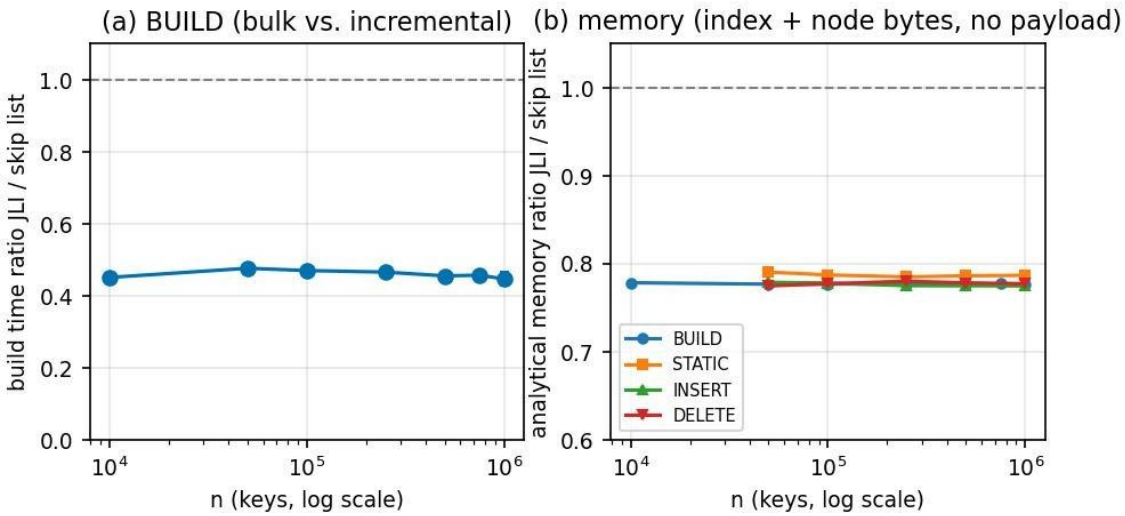


Figure 2. BUILD time ratio with 95 % intervals (a), and mean analytical memory ratio by class, counting index and node bytes without payload (b).

5.2 Static search

On machine A, JLI was faster in 25 of 30 configurations, slower in 2, and indistinguishable in 3, with ratios 0.81–1.23 (Table 6, Figure 3). Where JLI is faster, the gain ranges from a few percent to about 19 % (adversarial at 1M). The two slower configurations are adversarial at 50K and 100K (1.20 at 50K); sequential is not distinguishable from parity at 500K and 1M (1.06 at 1M). The repetition of the suite on machine A gave a similar picture (24 faster, 4 spanning 1, 2 slower).

On machine B, JLI was faster in 25 configurations, slower in 1 and indistinguishable in 4, with ratios 0.67–1.30. The dependence on n differs between the machines: for adversarial the ratio grows with n on machine B (0.67 at 50K, 1.30 at 1M) and falls on machine A (1.20 → 0.81). Across the 30 paired configurations the ratios are weakly negatively correlated (Pearson r = −0.31, Spearman ρ = −0.15; Figure 4). The data do not identify the cause; hardware differences such as cache hierarchy are a hypothesis we did not test.

Pattern	50K	100K	250K	500K	1M
Main machine					
random	0.94	0.92	0.88	0.86	0.87

sequential	0.88	0.91	0.93	1.04†	1.06†
Pattern	50K	100K	250K	500K	1M
hotspot	0.94	0.94	0.95	0.93	0.90
zipfian	0.91	0.91	1.01†	0.86	0.86
miss	0.95	0.96	0.96	0.91	0.94
adversarial	**1.20**	**1.23**	0.96	0.86	0.81
Second machine					
random	0.76	0.78	0.84	0.88	0.94
sequential	0.68	0.74	0.78	0.92	0.93
hotspot	0.80	0.80	0.80	0.83	0.80
zipfian	0.79	0.79	0.88	0.83	0.84
miss	0.78	0.79	0.91	0.91	0.98†
adversarial	0.67	0.82†	0.90†	1.15†	**1.30**

Table 6. STATIC latency ratio by pattern and size. Bold: JLI slower (interval above 1); †: interval spans 1.

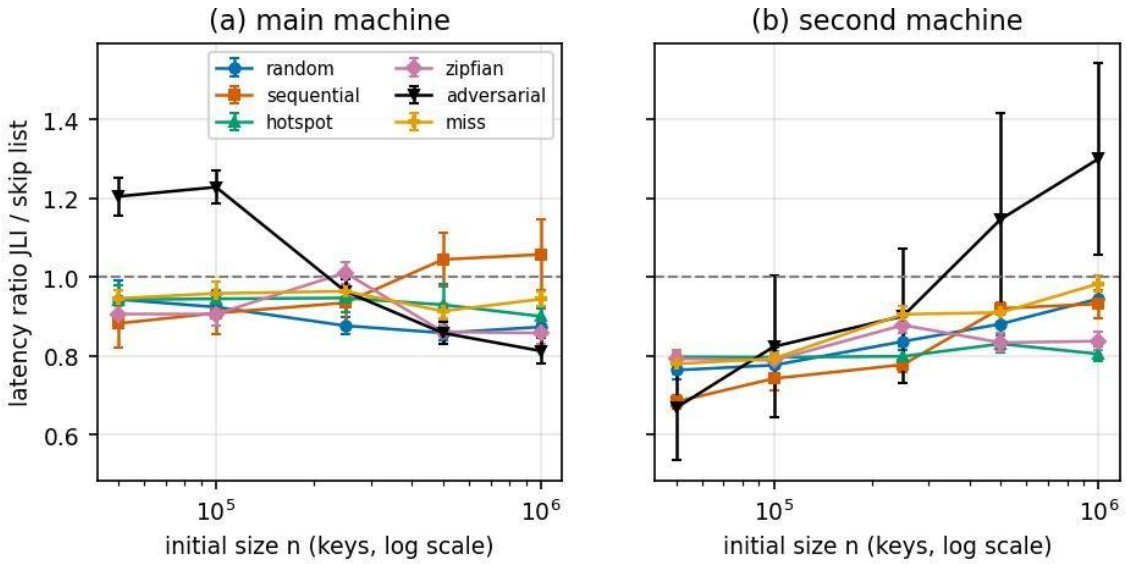


Figure 3. STATIC latency ratio (JLI/skip list) with 95 % intervals; n on a log axis. (a) Machine A. (b) Machine B.

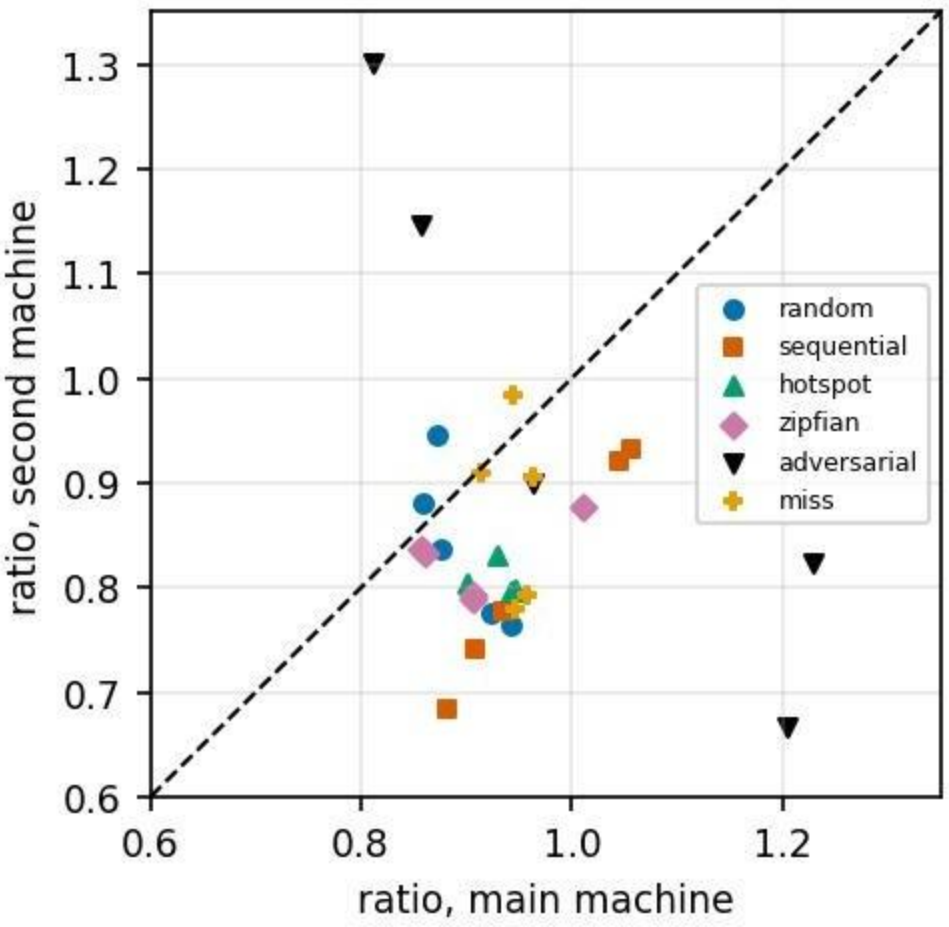


Figure 4. STATIC ratio on the two machines, one point per configuration; dashed line: equality.

5.3 Insertion and deletion

JLI was faster in 8 and slower in 17 of 25 INSERT configurations, and faster in 4, slower in 19 and indistinguishable in 2 of 25 DELETE configurations (Table 7, Figure 5). Its INSERT wins are the adversarial tail appends (all five sizes, ratio rising from 0.65 to 0.83 with n) and random insertion at 50K, 100K and 1M. Sequential, hotspot and zipfian insertion were slower in all sizes. For DELETE, JLI wins only for sequential at 100K–500K and random at 250K; deletion from the tail (adversarial) is the worst case, with the ratio rising from 1.31 at 50K to 2.05 at 1M. The rise with n for adversarial deletion is in the direction the informal analysis of Section 3.4 predicts. For adversarial insertion that analysis predicts O(1) amortised cost beyond location because of tail sealing, so it does not explain the rise from 0.65 to 0.83; we did not isolate the cause. Five sizes with size-dependent parameters cannot establish growth rates.

Pattern	50K	100K	250K	500K	1M
INSERT					
random	0.93	0.92	**1.09**	**1.02**	0.96

Pattern	50K	100K	250K	500K	1M
sequential	**1.01**	**1.06**	**1.06**	**1.22**	**1.39**
hotspot	**1.11**	**1.11**	**1.09**	**1.16**	**1.23**
zipfian	**1.11**	**1.08**	**1.06**	**1.07**	**1.11**
adversarial	0.65	0.69	0.75	0.78	0.83
DELETE					
random	**1.11**	**1.08**	0.97	1.02†	**1.14**
sequential	**1.08**	0.81	0.80	0.87	**1.03**
hotspot	**1.08**	**1.14**	**1.12**	**1.21**	**1.19**
zipfian	**1.07**	**1.03**	1.00†	**1.01**	**1.03**
adversarial	**1.31**	**1.42**	**1.54**	**1.62**	**2.05**

Table 7. INSERT and DELETE latency ratio on machine A. Bold: JLI slower; †: interval spans 1.

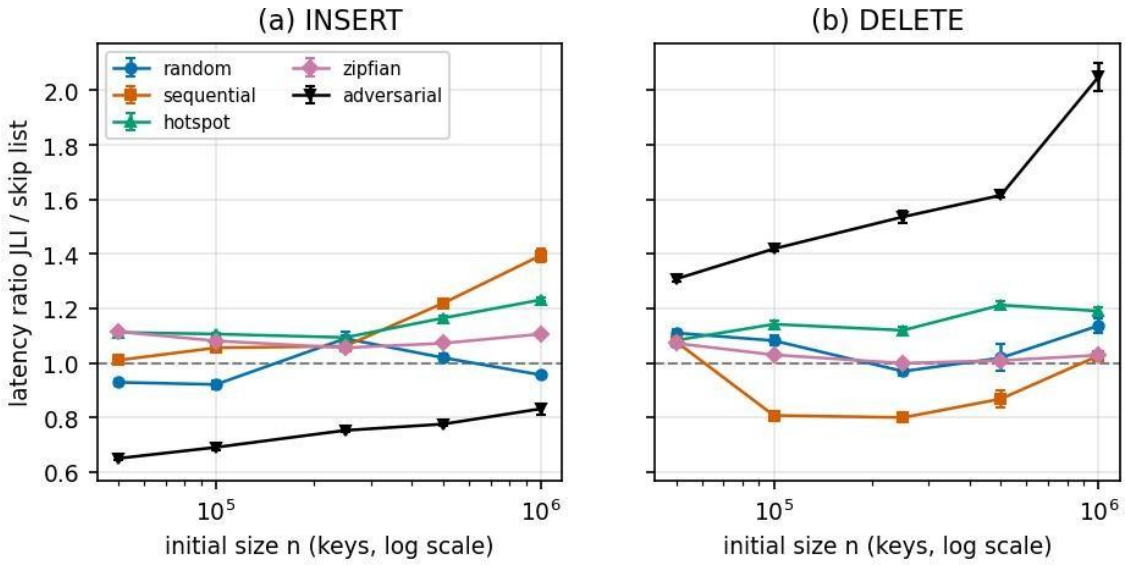


Figure 5. INSERT and DELETE latency ratio (JLI/skip list) with 95 % intervals.

5.4 Effect of maintenance on insertion

Table 8 and Figure 6 use the common grid of 10 cells (n = 50,000 and 100,000 at five run lengths each; Section 4.6). In these long runs on a growing structure, JLI was slower than the skip list for random, sequential, hotspot and zipfian insertion at every run length (mean ratios 1.11–1.47 with maintenance on), and faster for adversarial tail appends at every run length (0.72–0.86). Maintenance changes JLI's own latency by pattern (last block of Table 8). For zipfian insertion it lowers JLI's latency from 5× onwards (0.83–0.92) and raises it slightly at 1× (1.03); for random insertion it raises it at 1× and 5× (1.38 and 1.22), is roughly neutral at 20× (0.97) and lowers it at 40× and 50× (0.77 and 0.82); for hotspot insertion it lowers it at all run lengths (0.86–0.97); for sequential insertion it raises it up to 20× (1.21, 1.08, 1.05), is neutral at 40× (0.99) and lowers it slightly at 50× (0.95); for adversarial insertion it raises JLI's latency at every run length (1.09–1.34). Several curves are non-monotone, so we read the result as

a trade-off that depends on workload and run length, not a general rule.

Pattern	1×	5×	20×	40×	50×
JLI/skip-list INSERT latency ratio, maintenance ON					
random	1.20	1.27	1.22	1.18	1.21
sequential	1.11	1.33	1.39	1.43	1.43
hotspot	1.21	1.38	1.47	1.46	1.45
zipfian	1.18	1.23	1.27	1.27	1.27
adversarial	0.72	0.74	0.81	0.84	0.86
JLI/skip-list INSERT latency ratio, maintenance OFF					
random	0.98	1.21	1.44	1.51	1.49
sequential	0.90	1.28	1.48	1.49	1.52
hotspot	1.24	1.40	1.52	1.57	1.56
zipfian	1.16	1.30	1.42	1.49	1.50
adversarial	0.62	0.64	0.71	0.74	0.75
JLI latency, ON divided by OFF (below 1: maintenance helps)					
random	1.38	1.22	0.97	0.77	0.82
sequential	1.21	1.08	1.05	0.99	0.95
hotspot	0.97	0.92	0.91	0.86	0.87
zipfian	1.03	0.92	0.92	0.87	0.83
adversarial	1.09	1.18	1.24	1.20	1.34

Table 8. Maintenance comparison for INSERT, mean over n = 50,000 and n = 100,000. Operations per key 1×, 5×, 20×, 40× and 50× correspond to 50,000, 250,000, 1,000,000, 2,000,000 and 2,500,000 operations at n = 50,000, and to 100,000, 500,000, 2,000,000, 4,000,000 and 5,000,000 operations at n = 100,000.

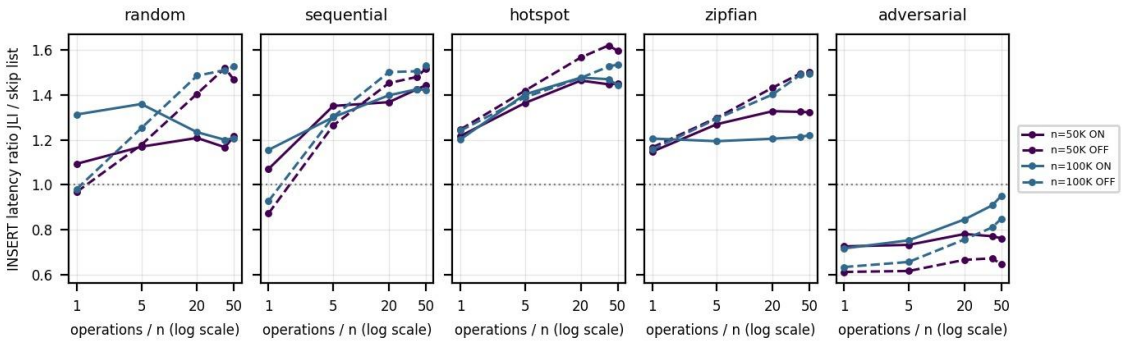


Figure 6. INSERT latency ratio against operations/n (log axis); solid: maintenance on, dashed: off; colour: n (common grid).

Figure 6 shows the same comparison by n. For random insertion the maintenance-off curves rise with operations/n at both n shown, while the maintenance-on curves stay flat or fall; at 20× the gap between the two arms (maintenance-off ratio minus maintenance-on ratio) is 0.25 at n = 100K and 0.20 at n = 50K. Two sizes are too few to estimate how the benefit scales with n.

5.5 Behaviour of the maintenance tiers

Section 5.4 measures what maintenance does to latency; this section reports how often each tier fires. Table 10 gives, for the maintenance-on arm of the insertion comparison, the number of local, region and global rebuilds, the nodes touched by the local and region rebuilds combined, and the dissolves, each per 1,000 mutations, read from the structure's own counters (these are maintained in the timed build and are not part of the step instrumentation that is compiled out). The tail-seal count and the split of node touches between the local and region tiers are not recorded in the benchmark output and are therefore not reported.

Pattern	Local rebuilds	Region rebuilds	Global rebuilds	Rebuild node touches (local + region)	Dissolves
random	0.36	0.44	0.00	7,815	0
sequential	0.00	0.50	0.00	1,247	0
hotspot	0.20	0.57	0.00	7,335	0
zipfian	0.03	0.45	0.03	789	0
adversarial	3.50	3.97	0.00	54,448	0

Table 10. Maintenance events and node touches per 1,000 mutations, maintenance on, at n = 100,000 and 50× operations per key (5,000,000 mutations); means over the 10 retained processes. Node touches are local and region rebuilds combined.

The escalation is designed so that the cheap tier absorbs most drift and the expensive tier is a rare escape hatch. The counters test that design directly: if it holds, local repairs should dominate the node

touches, region rebuilds should be less frequent, and global rebuilds should be rare outside the tail pattern; for adversarial tail appends, tail seals (about one per S appends) rather than region or global rebuilds are expected, and the maintenance cost seen in Table 8 for this pattern should then be scan overhead rather than repair work. Table 10 does not confirm the expected pattern. Region rebuilds were more frequent than local rebuilds in all five patterns (for example 0.57 against 0.20 per 1,000 mutations for hotspot, 0.50 against 0 for sequential, and 3.97 against 3.50 for adversarial), so the cheap tier does not absorb most of the drift by event count; whether local repairs dominate the node touches cannot be tested, because the counter combines the two tiers. Global rebuilds were rare, at most 0.03 per 1,000 mutations in this cell, and occurred in only one non-tail pattern here (zipfian); at n = 50,000 and 50× they also occurred for random and hotspot (0.04 per 1,000). Dissolves were zero for all patterns, as expected for an insert-only workload, since dissolving belongs to the delete path. The adversarial pattern differs from the others by roughly an order of magnitude: it had 3.50 local and 3.97 region rebuilds per 1,000 appends, against at most 0.36 and 0.57 for the other patterns, and about 54,000 rebuild node touches per 1,000 mutations against at most about 7,800, with no global rebuilds. The maintenance cost for tail appends in Table 8 is therefore not only scan overhead: the tiers repair repeatedly even with tail sealing active. Tail-seal counts were not recorded, so the expectation that seals rather than rebuilds absorb the tail drift could not be tested directly. Per 1,000 mutations, the rebuild rates of the non-tail patterns were lower at 50× than at 20× (for example region rebuilds for random 0.44 against 1.10, at n = 100,000), while the adversarial rates were unchanged (3.97 against 3.95); the former is consistent with the structure growing as mutations accumulate, but we did not test that explanation.

6 Discussion

What deferred maintenance buys. The maintenance on/off comparison (Section 5.4) is the closest we have to a direct test of the central design choice, which is to leave the index stale and repair it later. Unrepaired drift accumulates: with maintenance off, JLI's insertion latency relative to the skip list worsens with run length for all four non-tail patterns, for example random from 0.98 at 1× to 1.51 at 40×, sequential from 0.90 to 1.52 at 50×, hotspot from 1.24 to 1.57 at 40× and zipfian from 1.16 to 1.50 at 50× (Table 8).
With maintenance on, the same ratios stay in a narrower band for random (1.18–1.27) and zipfian (1.18–1.27), and plateau at about 1.4–1.5 for sequential and hotspot. At 40× and 50× operations per key the maintenance-on arm is faster than the maintenance-off arm for all four of these patterns (for sequential only marginally at 40×, 0.99).

Maintenance has a break-even horizon. Its cost is paid first and its benefit builds with run length. For random insertion the ratio of JLI's latency with maintenance on to off is 1.38 at 1×, 1.22 at 5×, 0.97 at 20× and 0.77 at 40×; zipfian crosses 1 between 1× and 5× (1.03, then 0.92); sequential is a net cost until 40× (1.21, 1.08, 1.05, then 0.99) and reaches 0.95 at 50×; hotspot benefits at every run length (0.86–0.97). A plausible reading is that the periodic scans run at fixed mutation intervals whether or not drift has appeared, so they are overhead on short runs, whereas repairs pay only once enough drift has accumulated. We did not separate scan cost from repair cost, so this is a hypothesis; Table 10 reports the tier counters that bear on it.

Tail workloads are the exception. For adversarial tail appends, maintenance raises JLI's own latency by 9–34 % at every run length, yet JLI is faster than the skip list in both arms (0.72–0.86 with maintenance on, 0.62–0.75 off). Tail sealing, which is active in both arms, was expected to absorb the drift that tail appends create, leaving little for the tiers to repair. Table 10 does not support this: adversarial appends

had the highest local and region rebuild rates of any pattern and about 54,000 rebuild node touches per 1,000 mutations (global rebuilds were zero), so the added cost under maintenance includes repair work, not only scan overhead. The deletion mirror has no sealing counterpart and is the worst result in Table 7, with the ratio rising from 1.31 to 2.05 as n grows.

The cost on mutation and search is usually small, but not in every regime. In the main suite (Table 7; maintenance on; ⌊ 2n/7⌋ operations), insertion latency was within 11 % of the skip list (ratio at most 1.11) in 21 of 25 configurations and deletion latency in 15 of 25. Static search was never more than 1.23× the skip list's latency and was faster in 25 of 30 configurations (Table 6). On the long, growing runs of Section 5.4 the gap is larger, 1.11–1.47 for the non-tail patterns with maintenance on, so "small" describes the main-suite regime and not every regime. Search latency was measured only on freshly built structures; we did not measure search after sustained mutation, and therefore make no claim about how much stale shortcuts lengthen the final walk when maintenance is off.

Possible headroom (hypothesis). The parameters were found by random search at a 500K reference size and transferred to the other sizes, and S, K and M do not scale with n. We conjecture that the reported results understate the design: a better understanding of the parameter landscape, for example a size-adaptive rule for S and K (the search bound is O(log n) only if K grows with S, Section 3.4) or a joint search over latency and memory (memory was never a tuning objective), could improve search and mutation while keeping the memory advantage. We have not tested this, and three observations limit what it can be expected to show. The results are already in-sample best cases for JLI against an untuned baseline, so further tuning on the same workloads has less room than it may appear, and a claim that tuned JLI beats the skip list needs held-out workloads. Part of the mutation cost is structural, namely the periodic scans and the dissolution of small segments, and tuning can shift it but not remove it. And the size dependence of the search ratio differs between the two machines, so a parameter rule tuned on one machine need not transfer. A direct test is to report the latency–memory Pareto front from the existing parameter-search data and to evaluate the selected parameters on held-out workloads and on both machines.

What the data support. The consistent finding is the memory reduction (75–80 %), which follows from the structure of the design and was observed in every configuration. The search advantage is small on machine A, is not stable across hardware, and depends on the pattern; we would not describe it as a general speed advantage. Mutation is where JLI usually loses, which is consistent with its design: it defers index repair, but the repair work and its scans still run, and tail deletion concentrates the shrinkage in one junction.

Interpretation limits. We observe associations between configuration and ratio; we did not run controlled experiments isolating individual mechanisms (for example, the cost of scans versus rebuilds, or cache effects). The explanations offered here and in Section 3.4 are hypotheses. The benchmark sizes (five sizes spanning a factor of 20) cannot confirm asymptotic claims.

Benchmark and tuning effects. Parameters were tuned per workload on the same machine and workloads as the evaluation, against an untuned baseline, so each JLI ratio is a best-case estimate. Absolute latencies are not stable across measurement campaigns on the same machine (the unchanged skip-list code varied by tens of percent between campaigns we compared), so we rely on within-run ratios, and small differences should be treated with caution. A degenerate miss pattern (all queries

above the maximum) short-circuits at the tail range check; we exclude it, and conclusions based on such misses would overstate JLI.

Practical reading. For workloads that are read-mostly, memory-constrained and not tail-heavy, the measurements indicate a memory saving with roughly neutral or slightly better search time on the tested machines. For insert- or delete-heavy workloads, especially at the tail, the skip list was usually faster. Maintenance is best treated as a tunable policy: in our runs it paid off on long non-tail insertion sequences and did not help tail appends.

7 Limitations and Threats to Validity

- **Single baseline.** Only a textbook skip list (p = 0.5) is compared. Other ordered indexes (B-trees, cache-conscious or concurrent skip lists, adaptive radix trees, learned indexes) are not evaluated.
- **In-sample tuning.** JLI parameters were tuned per workload against the baseline on the evaluation workloads; the baseline was not tuned. Results on unseen workloads may be worse.
- **Workloads.** Synthetic patterns with 32-bit integer keys, no payload, single-threaded execution, sizes from 10K to 1M. Real datasets, variable-length keys, mixed read/write workloads and concurrency are not covered.
- **Hardware and software.** Two Windows machines, one compiler configuration **[CONFIRM: GCC version and flags]**; results may be compiler-dependent.
- **Statistical power.** Ten retained processes per configuration. Intervals are descriptive; after multiplicity correction no cell reaches the conventional significance level because the exact test cannot, by design. Batch-mean latencies are reported, not per-operation tail latency.
- **Memory.** Analytical and payload-free; resident-set size and total memory with payloads were not measured.
- **BUILD.** Compares bulk construction with incremental insertion.
- **Complexity.** The amortised analysis is informal and workload-dependent; no proof of an amortised bound is given.
- **Maintenance comparison.** Both arms use parameters tuned for maintenance on; results are limited to the common grid and the sizes and run lengths reported.
- **Search after mutation.** Search was timed on freshly built structures only. Search latency after sustained mutation, with maintenance on or off, was not measured, and the separate costs of scans and repairs within maintenance were not isolated.
- **Cross-machine inconsistency.** The size dependence of the search ratio differs between the two machines and is unexplained.

8 Conclusion and Future Work

The Junction Linked List moves all index data out of the nodes of a sorted list and repairs the index lazily. Against a textbook skip list in the same harness, this design used 75–80 % of the analytical memory in all 87 configurations, built 2.1–2.2× faster than incremental insertion, and searched faster in 25 of 30 configurations on the main machine, although the size dependence of the search ratio did not replicate on a second machine.

The cost of the design is on mutation. Insertion and deletion were usually slower than the skip list, but within about 11 % of it in 21 of 25 insertion and 15 of 25 deletion configurations at the main-suite run

lengths, and the cost is larger on long growing runs. The maintenance comparison shows why the tiered repair matters: with it off, JLI's insertion latency drifts to about 1.5× the skip list's on long runs for four of five patterns; with it on, the same patterns settle lower, and for random, zipfian and hotspot insertion maintenance lowers JLI's own latency by 23 %, 13 % and 14 % respectively at 40× the initial size.
Maintenance has a break-even horizon, it is a net cost on short runs, and it does not help tail appends. Tail-end workloads, and tail deletion in particular, remain the failure case. These findings come from a single baseline, in-sample tuning and two machines.

Future work includes comparison with B-trees, cache-conscious skip lists and other ordered indexes; held-out workloads without per-workload tuning; measurement of search after sustained mutation with maintenance on and off; a parameter-sensitivity analysis, the latency–memory Pareto front and size-adaptive parameters (the hypothesis above); controlled experiments separating scan cost from repair cost and isolating cache effects; resident-set and payload-inclusive memory; concurrency; and a formal amortised analysis or a maintenance policy that removes the tail-workload failure case.

Acknowledgements

The structure is also known as the AMJ List (Anna Mani Junction List), named in tribute to the physicist Anna Mani (1918–2001). A pioneering Indian physicist and meteorologist, Mani is celebrated for helping India achieve self-sufficiency in weather forecasting by designing and manufacturing its own meteorological instruments, including the first Indian-made ozonesonde. Her later work laid the foundation for the country's renewable energy assessments through extensive solar and wind measurement surveys. This dedication honors her legacy of structural self-reliance and her foundational contributions to scientific measurement.

Appendix A. Reproducibility

All tables, figures and numbers in this paper are generated from the raw per-process benchmark CSVs by an analysis script and a table generator provided with the supplementary material; the aggregated CSVs produced by the benchmark drivers are used only as cross-checks. The benchmark source (jli_v3_1.c, 8_1.c), the drivers and the raw data are provided as supplementary material [https://github.com/mehal17chaudhary-sudo/Junction-Linked-List_V2] . Build line: gcc -O2 -o bench.exe 8.c -lm.

Appendix B. All main-suite configurations

Table 9 lists every main-suite configuration with the 95 % interval of the mean ratio and the raw and Holm-adjusted Wilcoxon p-values.

Class	Pattern	n	Ratio [95 % CI]	Raw p	Holm p
BUILD	sorted	10,000	0.450 [0.437, 0.463]	0.0020	0.170
BUILD	sorted	50,000	0.475 [0.467, 0.483]	0.0020	0.170

BUILD	sorted	100,000	0.469 [0.463, 0.475]	0.0020	0.170
Class	**Pattern**	**n**	**Ratio [95 % CI]**	**Raw p**	**Holm p**
BUILD	sorted	250,000	0.465 [0.454, 0.475]	0.0020	0.170
BUILD	sorted	500,000	0.454 [0.447, 0.461]	0.0020	0.170
BUILD	sorted	750,000	0.456 [0.447, 0.466]	0.0020	0.170
BUILD	sorted	1,000,000	0.446 [0.426, 0.465]	0.0020	0.170
DELETE	adversarial	50,000	1.308 [1.299, 1.318]	0.0020	0.170
DELETE	adversarial	100,000	1.419 [1.410, 1.428]	0.0020	0.170
DELETE	adversarial	250,000	1.536 [1.514, 1.558]	0.0020	0.170
DELETE	adversarial	500,000	1.615 [1.607, 1.624]	0.0020	0.170
DELETE	adversarial	1,000,000	2.050 [1.999, 2.102]	0.0020	0.170
DELETE	hotspot	50,000	1.083 [1.071, 1.095]	0.0020	0.170
DELETE	hotspot	100,000	1.142 [1.129, 1.155]	0.0020	0.170
DELETE	hotspot	250,000	1.120 [1.108, 1.132]	0.0020	0.170
DELETE	hotspot	500,000	1.212 [1.198, 1.227]	0.0020	0.170
DELETE	hotspot	1,000,000	1.191 [1.176, 1.206]	0.0020	0.170
DELETE	random	50,000	1.110 [1.095, 1.126]	0.0020	0.170
DELETE	random	100,000	1.082 [1.064, 1.100]	0.0020	0.170
DELETE	random	250,000	0.970 [0.953, 0.986]	0.0098	0.170
DELETE	random	500,000	1.019 [0.969, 1.068]	0.4316	1.000

DELETE	random	1,000,000	1.137 [1.110, 1.163]	0.0020	0.170
Class	**Pattern**	**n**	**Ratio [95 % CI]**	**Raw p**	**Holm p**
DELETE	sequential	50,000	1.078 [1.064, 1.093]	0.0020	0.170
DELETE	sequential	100,000	0.808 [0.798, 0.817]	0.0020	0.170
DELETE	sequential	250,000	0.800 [0.794, 0.806]	0.0020	0.170
DELETE	sequential	500,000	0.868 [0.835, 0.901]	0.0020	0.170
DELETE	sequential	1,000,000	1.027 [1.018, 1.036]	0.0020	0.170
DELETE	zipfian	50,000	1.072 [1.058, 1.086]	0.0020	0.170
DELETE	zipfian	100,000	1.030 [1.018, 1.041]	0.0020	0.170
DELETE	zipfian	250,000	0.999 [0.988, 1.010]	1.0000	1.000
DELETE	zipfian	500,000	1.009 [1.004, 1.014]	0.0020	0.170
DELETE	zipfian	1,000,000	1.028 [1.017, 1.040]	0.0020	0.170
INSERT	adversarial	50,000	0.650 [0.643, 0.657]	0.0020	0.170
INSERT	adversarial	100,000	0.691 [0.679, 0.703]	0.0020	0.170
INSERT	adversarial	250,000	0.753 [0.746, 0.759]	0.0020	0.170
INSERT	adversarial	500,000	0.776 [0.768, 0.783]	0.0020	0.170
INSERT	adversarial	1,000,000	0.832 [0.812, 0.852]	0.0020	0.170
INSERT	hotspot	50,000	1.112 [1.106, 1.119]	0.0020	0.170
INSERT	hotspot	100,000	1.106 [1.101, 1.111]	0.0020	0.170
INSERT	hotspot	250,000	1.094 [1.084, 1.103]	0.0020	0.170

INSERT	hotspot	500,000	1.165 [1.157, 1.173]	0.0020	0.170
Class	**Pattern**	**n**	**Ratio [95 % CI]**	**Raw p**	**Holm p**
INSERT	hotspot	1,000,000	1.232 [1.225, 1.238]	0.0020	0.170
INSERT	random	50,000	0.929 [0.917, 0.941]	0.0020	0.170
INSERT	random	100,000	0.921 [0.906, 0.936]	0.0020	0.170
INSERT	random	250,000	1.090 [1.065, 1.114]	0.0020	0.170
INSERT	random	500,000	1.019 [1.007, 1.032]	0.0098	0.170
INSERT	random	1,000,000	0.956 [0.951, 0.962]	0.0020	0.170
INSERT	sequential	50,000	1.010 [1.001, 1.019]	0.0195	0.176
INSERT	sequential	100,000	1.056 [1.044, 1.068]	0.0020	0.170
INSERT	sequential	250,000	1.062 [1.054, 1.069]	0.0020	0.170
INSERT	sequential	500,000	1.219 [1.207, 1.230]	0.0020	0.170
INSERT	sequential	1,000,000	1.395 [1.370, 1.419]	0.0020	0.170
INSERT	zipfian	50,000	1.115 [1.103, 1.126]	0.0020	0.170
INSERT	zipfian	100,000	1.081 [1.073, 1.089]	0.0020	0.170
INSERT	zipfian	250,000	1.056 [1.050, 1.061]	0.0020	0.170
INSERT	zipfian	500,000	1.072 [1.066, 1.078]	0.0020	0.170
INSERT	zipfian	1,000,000	1.106 [1.103, 1.110]	0.0020	0.170
STATIC	adversarial	50,000	1.205 [1.157, 1.252]	0.0020	0.170
STATIC	adversarial	100,000	1.228 [1.186, 1.270]	0.0020	0.170

STATIC	adversarial	250,000	0.964 [0.932, 0.995]	0.0195	0.176
Class	**Pattern**	**n**	**Ratio [95 % CI]**	**Raw p**	**Holm p**
STATIC	adversarial	500,000	0.858 [0.830, 0.886]	0.0020	0.170
STATIC	adversarial	1,000,000	0.812 [0.780, 0.844]	0.0020	0.170
STATIC	hotspot	50,000	0.942 [0.907, 0.978]	0.0098	0.170
STATIC	hotspot	100,000	0.945 [0.923, 0.966]	0.0020	0.170
STATIC	hotspot	250,000	0.947 [0.912, 0.982]	0.0137	0.170
STATIC	hotspot	500,000	0.929 [0.877, 0.982]	0.0020	0.170
STATIC	hotspot	1,000,000	0.900 [0.881, 0.919]	0.0020	0.170
STATIC	miss	50,000	0.945 [0.925, 0.965]	0.0020	0.170
STATIC	miss	100,000	0.958 [0.927, 0.989]	0.0137	0.170
STATIC	miss	250,000	0.964 [0.950, 0.977]	0.0020	0.170
STATIC	miss	500,000	0.913 [0.895, 0.932]	0.0020	0.170
STATIC	miss	1,000,000	0.944 [0.925, 0.963]	0.0020	0.170
STATIC	random	50,000	0.943 [0.895, 0.992]	0.0371	0.260
STATIC	random	100,000	0.924 [0.898, 0.949]	0.0020	0.170
STATIC	random	250,000	0.876 [0.856, 0.897]	0.0020	0.170
STATIC	random	500,000	0.858 [0.840, 0.876]	0.0020	0.170
STATIC	random	1,000,000	0.873 [0.851, 0.896]	0.0020	0.170
STATIC	sequential	50,000	0.882 [0.821, 0.943]	0.0098	0.170

STATIC	sequential	100,000	0.908 [0.856, 0.961]	0.0098	0.170
Class	**Pattern**	**n**	**Ratio [95 % CI]**	**Raw p**	**Holm p**
STATIC	sequential	250,000	0.935 [0.871, 0.998]	0.1055	0.633
STATIC	sequential	500,000	1.044 [0.975, 1.113]	0.2324	1.000
STATIC	sequential	1,000,000	1.057 [0.967, 1.147]	0.3750	1.000
STATIC	zipfian	50,000	0.906 [0.882, 0.930]	0.0020	0.170
STATIC	zipfian	100,000	0.906 [0.876, 0.935]	0.0020	0.170
STATIC	zipfian	250,000	1.011 [0.982, 1.039]	0.7695	1.000
STATIC	zipfian	500,000	0.861 [0.830, 0.891]	0.0020	0.170
STATIC	zipfian	1,000,000	0.858 [0.829, 0.887]	0.0020	0.170

Table 9. Per-configuration statistics for the main suite (machine A): mean ratio with 95 % interval, raw and Holm-adjusted Wilcoxon p.

Appendix C. Maintenance comparison, all completed cells

With the benchmark limited to n = 50,000 and n = 100,000, the completed (pattern, n, operations) cells are exactly the 10 cells of the common grid used in Section 5.4. The per-cell INSERT latency ratios for these cells are therefore the series plotted in Figure 6, and their means over the two sizes are given in Table 8.

Appendix D. Literature search

The search used OpenAlex (reference resolution and keyword and semantic queries) plus targeted web checks of four records; only the top of each ranked list was screened. Two candidate references that

could not be verified (a SIGMOD 2010 tree-search paper and a PODC 2013 concurrent search structure) were excluded. The queries and the resulting selections are listed in the supplementary search log.

References

[1] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Communications of the ACM*, 1990, doi: 10.1145/78973.78977.

[2] J. I. Munro, T. Papadakis, and R. Sedgewick, "Deterministic skip lists," in *Proceedings of the annual ACM-SIAM symposium on discrete algorithms (SODA)*, 1992.

[3] A. Bagchi, A. L. Buchsbaum, and M. T. Goodrich, "Biased skip lists," *Algorithmica*, 2005, doi: 10.1007/s00453-004-1138-6.

[4] D. Golovin, "The B-skip-list: A simpler uniquely represented alternative to B-trees." 2010. doi: 10.48550/arxiv.1005.0662.

[5] S. Cho and S. K. Sahni, "Weight-biased leftist trees and modified skip lists," *ACM Journal of Experimental Algorithmics*, 1998, doi: 10.1145/297096.297111.

[6] M. A. Bender, M. Farach-Colton, R. Johnson, S. Mauras, T. Mayer, *et al.*, "Write-optimized skip lists," in *ACM SIGMOD-SIGACT-SIGAI symposium on principles of database systems (PODS)*, 2017. doi: 10.1145/3034786.3056117.

[7] M. P. Herlihy, Y. Lev, V. Luchangco, and N. Shavit, "A simple optimistic skiplist algorithm," in *Structural information and communication complexity (SIROCCO)*, 2007. doi: 10.1007/978-3-540-72951-8_11.

[8] T. Crain, V. Gramoli, and M. Raynal, "No hot spot non-blocking skip list," in *IEEE international conference on distributed computing systems (ICDCS)*, 2013. doi: 10.1109/icdcs.2013.42.

[9] Z. Xie, Q. Cai, H. V. Jagadish, B. C. Ooi, and W.-F. Wong, "Parallelizing skip lists for in-memory multi-core database systems," in *IEEE international conference on data engineering (ICDE)*, 2017. doi: 10.1109/icde.2017.54.

[10] M. R. Brown and R. E. Tarjan, "A representation for linear lists with movable fingers," in *ACM symposium on theory of computing (STOC)*, 1978. doi: 10.1145/800133.804328.

[11] L. J. Guibas, E. M. McCreight, M. F. Plass, and J. R. Roberts, "A new representation for linear lists," in *ACM symposium on theory of computing (STOC)*, 1977. doi: 10.1145/800105.803395.

[12] S. Huddleston and K. Mehlhorn, "A new data structure for representing sorted lists," *Acta Informatica*, 1982, doi: 10.1007/bf00288968.

[13] S. R. Kosaraju, "Localized search in sorted lists," in *ACM symposium on theory of computing (STOC)*, 1981. doi: 10.1145/800076.802458.

[14] P. F. Dietz, "Maintaining order in a linked list," in *ACM symposium on theory of computing (STOC)*, 1982. doi: 10.1145/800070.802184.

[15] R. Bayer and E. M. McCreight, “Organization and maintenance of large ordered indexes,” *Acta Informatica*, 1972, doi: 10.1007/bf00288683.

[16] J. Rao and K. A. Ross, “Cache conscious indexing for decision-support in main memory,” in *International conference on very large data bases (VLDB)*, 1999. doi: 10.7916/d8t441zb.

[17] J. Rao and K. A. Ross, “Making B+-trees cache conscious in main memory,” in *ACM SIGMOD international conference on management of data*, 2000. doi: 10.1145/335191.335449.

[18] M. A. Bender, E. D. Demaine, and M. Farach-Colton, “Cache-oblivious B-trees,” *SIAM Journal on Computing*, vol. 35, no. 2, pp. 341–358, 2005, doi: 10.1137/S0097539701389956.

[19] K. Platz, N. Mittal, and S. Venkatesan, “Practical concurrent unrolled linked lists using lazy synchronization,” *Journal of Parallel and Distributed Computing*, 2019, doi: 10.1016/j.jpdc.2019.11.005.

[20] V. Leis, A. Kemper, and T. Neumann, “The adaptive radix tree: ARTful indexing for main-memory databases,” in *Proceedings of the IEEE international conference on data engineering (ICDE)*, 2013. doi: 10.1109/icde.2013.6544812.

[21] I. Galperin and R. L. Rivest, “Scapegoat trees,” in *Proceedings of the annual ACM-SIAM symposium on discrete algorithms (SODA)*, 1993. doi: 10.5555/313559.313676.

[22] D. D. Sleator and R. E. Tarjan, “Self-adjusting binary search trees,” *Journal of the ACM*, 1985, doi: 10.1145/3828.3835.

[23] M. A. Bender, M. Farach-Colton, and M. A. Mosteiro, “Insertion sort is O($n \log n$),” *Theory of Computing Systems*, 2006, doi: 10.1007/s00224-005-1237-z.

[24] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, “The case for learned index structures,” in *Proceedings of the ACM SIGMOD international conference on management of data*, 2018. doi: 10.1145/3183713.3196909.

[25] P. Ferragina and G. Vinciguerra, “The PGM-index: A fully-dynamic compressed learned index with provable worst-case bounds,” *Proceedings of the VLDB Endowment*, vol. 13, no. 8, pp. 1162–1175, 2020, doi: 10.14778/3389133.3389135.

[26] R. Marcus, A. Kipf, A. van Renen, M. Stoian, S. Misra, *et al.*, “Benchmarking learned indexes,” *Proceedings of the VLDB Endowment*, 2020, doi: 10.14778/3421424.3421425.

[27] C. Wongkham, B. Lu, C. Liu, Z. Zhong, E. Lo, *et al.*, “Are updatable learned indexes ready?” *Proceedings of the VLDB Endowment*, 2022, doi: 10.14778/3551793.3551848.

[28] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *ACM SIGPLAN conference on object-oriented programming, systems, languages, and applications (OOPSLA)*, 2007. doi: 10.1145/1297027.1297033.

[29] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in *International symposium on memory management (ISMM)*, 2013. doi: 10.1145/2464157.2464160.

[30] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!” in *International conference on architectural support for programming languages and operating systems (ASPLOS)*, 2009. doi: 10.1145/1508244.1508275.

[31] T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems,” in *International conference for high performance computing, networking, storage and analysis (SC)*, 2015. doi: 10.1145/2807591.2807644.